

Lightning lesson, sixty minutes

Build Your First Software Factory Execution Harness

Hands on. Attendees follow along in their own repository and finish with an isolated place for agent work.

Twenty minutes ahead

Setup you do alone

Goal Worktrees can be created and removed instantly, with nothing left over from an earlier rehearsal.

Say Nothing. Preparation only.

Run

```
$ git worktree list
$ make clean
$ make demo STEP=02
```

Check

- Only your main checkout is listed before you begin.
- A rehearsal leaves nothing behind, so the count is the same afterwards.

Ask *Nobody. Widen the terminal font now, because this session is watched closely.*

0:00 to 0:10

Why your checkout is the wrong place

Goal	The room stops seeing isolation as ceremony and starts seeing it as the fix for something that already bit them.
Say	• Collect the war stories first. Ask who has had an agent commit something half-finished, or write into the branch they had just switched to. • Reframe each story as a missing place rather than a misbehaving tool. • State what you are about to build and what it will cost: about a second per task, and a directory you can look inside when something goes wrong.
Run	Terminal open, nothing typed.
Room sees	An empty prompt and their own stories on the whiteboard.
Ask	<i>Who has lost work to a background run? What did it cost to recover?</i>

0:10 to 0:28

A second checkout, built live

Goal Everyone has created one, written a file in it, and confirmed their own branch never moved.

- Say**
- Explain the object store once: one repository, several checkouts, no duplicated history and no second clone to keep current.
 - Build it in front of them rather than opening the finished file. Create, list, write, remove.
 - Call out the removal explicitly. A design that is hard to undo does not get adopted, and this one is a single command.
-

Run

```
$ node steps/02-execution/demo.ts create 142
$ git worktree list
$ git status
$ node steps/02-execution/demo.ts remove 142
```

The implementation is `steps/02-execution/worktree.ts`, and it is short enough to read on screen.

- Room sees**
- Two entries in the worktree list, then one again.
 - A clean status in the main checkout throughout, which is the whole point.
-

Ask *Your build needs installed dependencies. Where do those live now, and what does that cost per task?*

0:28 to 0:42

Two runs, one item

Goal They understand how ownership is settled without anything new to operate.

Say

- Pose the collision before showing the answer. Two scheduled runs wake up, both see the same item, both start.
- Let someone propose a lock, then price it honestly: a service, its availability, and what happens when a holder dies.
- Reveal the alternative and let the surprise land. The name is derived from the item, so the loser finds out by being rejected.

Run

```
$ node steps/02-execution/demo.ts race 142
```

Room sees

- Two runs starting together on the same item.
- One continuing, one stopping with the rejected push printed underneath.

Ask *What else in your pipeline would be simpler if the coordination lived in a tool you already run?*

0:42 to 0:54

What it can reach from in there

Goal Attendees can state limits for one task in their own repository and know what happens when one is exceeded.

- Say**
- Separate the two questions cleanly. Isolation answered where. Limits answer what it can touch once it is there.
 - Argue for declaring limits per task rather than globally, using the lockfile example: that task genuinely needs the network and most do not.
 - Insist on the ending. Exceeding a limit stops the run. A warning in a log is not a limit.
-

Run

```
$ node steps/02-execution/demo.ts limits
```

Declared in `steps/02-execution/limits.ts`, where a reviewer can read them.

- Room sees**
- A permitted write succeeding.
 - A write outside the allowed set refused, naming the path and the rule.
 - A command not on the list refused before it runs.
-

Ask *List three commands you would never want a background run to execute. Why those three?*

0:54 to 1:00

Take it home

Goal	Everyone has the harness working against their own repository before they close the call.
Say	• Give the one instruction to try tonight, and say it slowly enough to be written down. • Warn about the dependency cost up front so it is expected rather than discovered.
Run	Leave the worktree list and a clean status side by side on screen.
Room sees	The repository address and the two files worth reading first.
Ask	<i>Any repository where a second checkout would not work? Bring it to the next session.</i>

0:35 to 0:45

Somebody else's sandbox, same rule

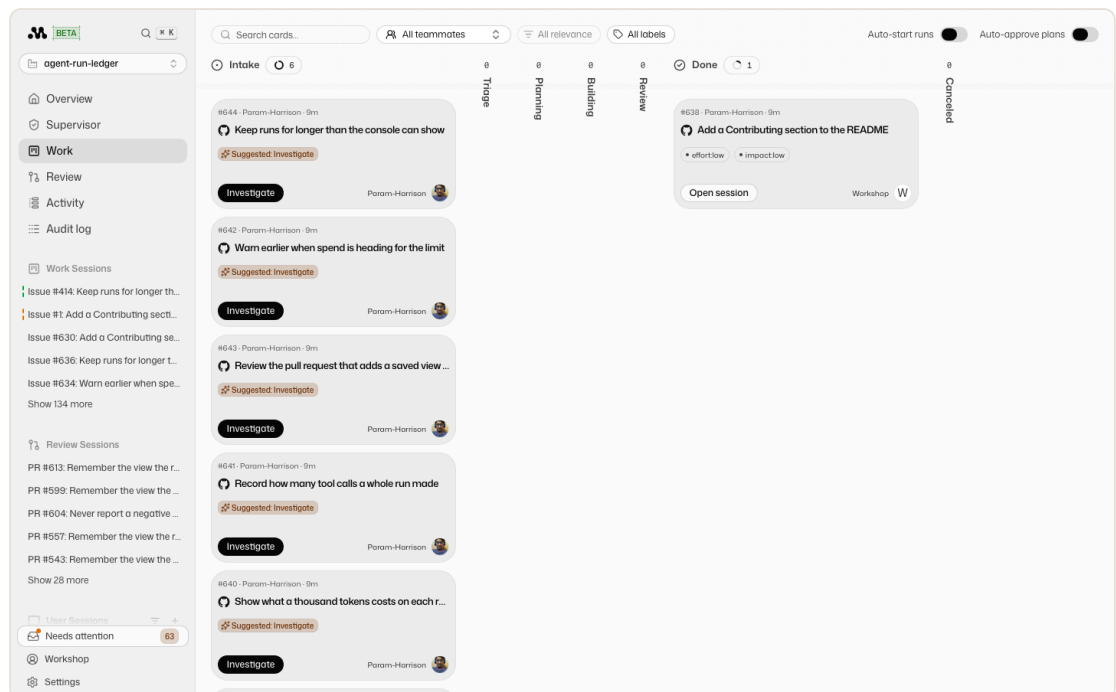
Goal They can say why an isolated checkout is a property of the harness rather than a courtesy.

Say

- Two harnesses now: the one built in this lesson and the one inside the product.
- Both give a run its own tree, and for the same reason, which is that a shared tree makes two runs indistinguishable when one of them breaks.
- Show where the checkout lives, and that deleting it costs nothing.

Run

```
$ ls ../mastra/.sandboxes || true
```



A run holds a checkout of its own. The repository on your disk is untouched while it works.

Show on the board

Work to a card in Building to its session to the file tree

A checkout of its own, every time

The session holds a clone under `.sandboxes`. Delete it while the run is going and the run dies; delete your own working tree and nothing happens, which is the whole point of the boundary.

The one that cost me an evening

A review once checked a pull request out in the Factory server's own directory rather than a sandbox. docker-compose.yml and package.json vanished and the only symptom was "compose file is invalid". Ask the room where they would have looked first.

Sandboxes are cattle

They are named by run, not by branch, and nothing in them is worth keeping once the pull request exists.

Room does

- Opens the sandbox directory and confirms it is a real clone.
- Confirms their own working tree has no new branch.

Ask

What breaks first if two runs share one checkout: the build, the test, or the diff?

Concepts: worktree per task, the branch as claim, limits, executor interface.